

2010376103 CSE4261 Assignment-13

OMOR NILOY

August 2025

Dataset Preparation

In all experiments related to synthetic face generation, the **CelebA** dataset was employed. CelebA consists of over 200,000 celebrity face images annotated with various attributes. For this work, the original images were resized to 64×64 pixels and normalized to the range $[-1, 1]$ to match the DCGAN input specifications.

- **Dataset:** CelebA dataset containing over 200,000 celebrity face images with attribute annotations.
- **Preprocessing:** Images resized to 64×64 pixels and normalized to $[-1, 1]$.
- **Subset Selection:** Randomly selected 10% of the dataset (approximately 20,000 images) using a fixed random seed (42) for reproducibility.
- **Data Splitting:**
 - Training set: 80% of the subset, used to train generator and discriminator.
 - Validation set: 10%, used to monitor overfitting and tune hyperparameters.
 - Test set: 10%, held out for final evaluation.
- **Data Loading:** PyTorch `DataLoader` with batch size 128; parallelized using multiple CPU workers for efficiency.
- **Data Augmentation:** No augmentation applied to preserve the original data distribution for adversarial learning.

For Conditional GAN (Long Hair Faces):

- **Dataset:** Subset of CelebA filtered for hair length attribute, labeling images as *long hair* (class 1) or *not long hair* (class 0).
- **Preprocessing:** Images resized to 64×64 pixels and normalized to $[-1, 1]$.
- **Subset Selection:** Random sample of 5,000 images selected using fixed random seed to ensure reproducibility.
- **Data Splitting:**
 - Training set: 70% of the subset, used to train the CGAN.
 - Validation set: 15%, used to monitor training progress and stability.
 - Test set: 15%, used for unbiased evaluation of generated images.
- **Data Loading:** PyTorch `DataLoader` with batch size 64, utilizing multiple workers for parallel data loading.
- **Data Augmentation:** None applied, to maintain consistency of class labels during conditional training.

1 Generating Synthetic Faces using Deep Convolutional GAN

[Github code link](#)

1.1 Model Description

In this task, we implemented a **Deep Convolutional Generative Adversarial Network (DCGAN)** to generate synthetic human faces using the CelebA dataset. The DCGAN consists of two main components:

- **Generator (G):** Takes a random noise vector from a latent space and progressively upsamples it through a series of transposed convolutional layers to produce realistic 64×64 RGB face images.
- **Discriminator (D):** A convolutional neural network that classifies input images as either real (from the dataset) or fake (generated by G).

The model follows the architectural guidelines proposed in the original DCGAN paper, with batch normalization and non-linear activations (**ReLU** for the generator, **LeakyReLU** for the discriminator) to stabilize training.

1.2 Dataset and Preprocessing

The **CelebA** dataset, containing aligned celebrity face images, was used. Preprocessing steps included:

- Resizing images to 64×64 pixels.
- Center cropping to maintain facial alignment.
- Normalizing pixel values to the range $[-1, 1]$.

For faster training, a random 10% subset of the dataset was used.

1.3 Training Setup

The DCGAN was trained for **50 epochs** with the following configuration:

- **Batch size:** 128
- **Latent vector size (z):** 100
- **Optimizer:** Adam ($\beta_1 = 0.5, \beta_2 = 0.999$)
- **Learning rate:** 0.0002
- **Loss function:** Binary Cross Entropy (BCE) loss for both G and D .

During each training iteration:

1. The discriminator was trained to maximize the probability of correctly classifying real and fake images.
2. The generator was trained to produce images that the discriminator classifies as real.

1.4 Results

The generator gradually improved at producing realistic faces, while the discriminator learned to distinguish between real and fake examples. A qualitative comparison of real and generated images is shown in Figure 1, where the DCGAN produces convincing facial features such as eyes, mouth, and hair structure.



Figure 1: Comparison between real and synthetic face images generated by DCGAN.

The evolution of the generator and discriminator losses over training is illustrated in Figure 4, highlighting the adversarial learning process and relative stability achieved during training.

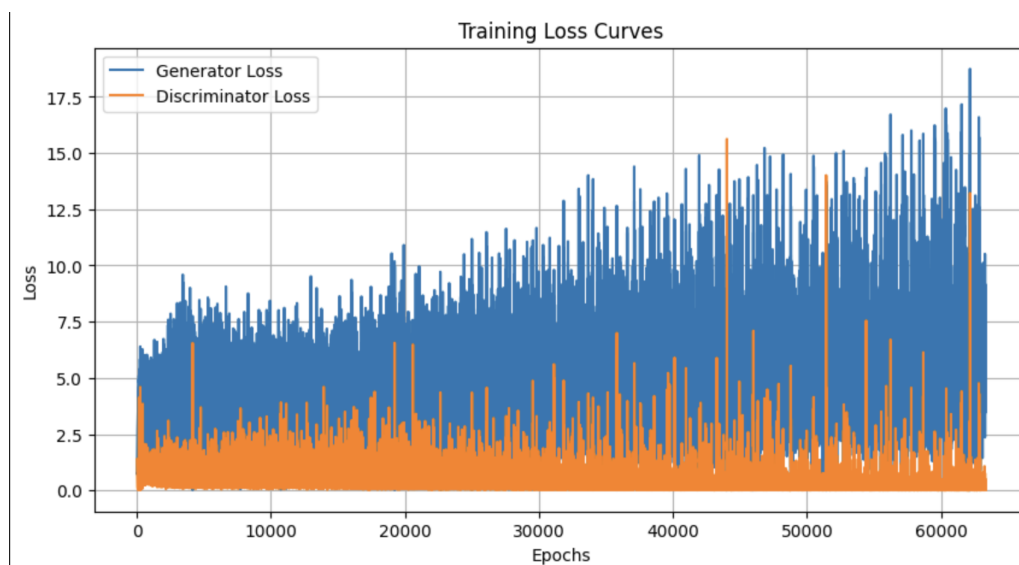


Figure 2: Generator and Discriminator loss curves during DCGAN training.

1.5 Conclusion

The DCGAN successfully learned to generate realistic synthetic face images from random noise vectors. While the generated images captured key facial features such as eyes, nose, and mouth, some outputs still contained artifacts, suggesting that further training or architectural enhancements could improve realism.

2 Generating Synthetic Faces with Long Hair using Conditional GAN

[Github code link](#)

2.1 Model and Dataset

In this work, we implemented a **Conditional Generative Adversarial Network (CGAN)** to generate synthetic face images conditioned on the presence of long hair, using the CelebA dataset.

A custom dataset class was designed to label images as *long hair* (class 1) or *not long hair* (class 0) based on CelebA attributes. Images were resized and normalized to 64×64 pixels with pixel values scaled between -1 and 1 .

The CGAN consists of a generator and discriminator, both conditioned on the hair-length class label. The generator takes a noise vector concatenated with an embedding of the class label to generate images, while the discriminator jointly evaluates image-label pairs to distinguish real from fake samples.

2.2 Training Setup

The model was trained for 150 epochs using the Adam optimizer ($\text{lr} = 2 \times 10^{-4}$, $\beta_1 = 0.5$) and Binary Cross Entropy loss. A batch size of 64 was used.

2.3 Results

Figure 3 shows a side-by-side comparison of real long hair images and the synthetic faces generated by the CGAN after training, demonstrating that the model captures key features of long hair.

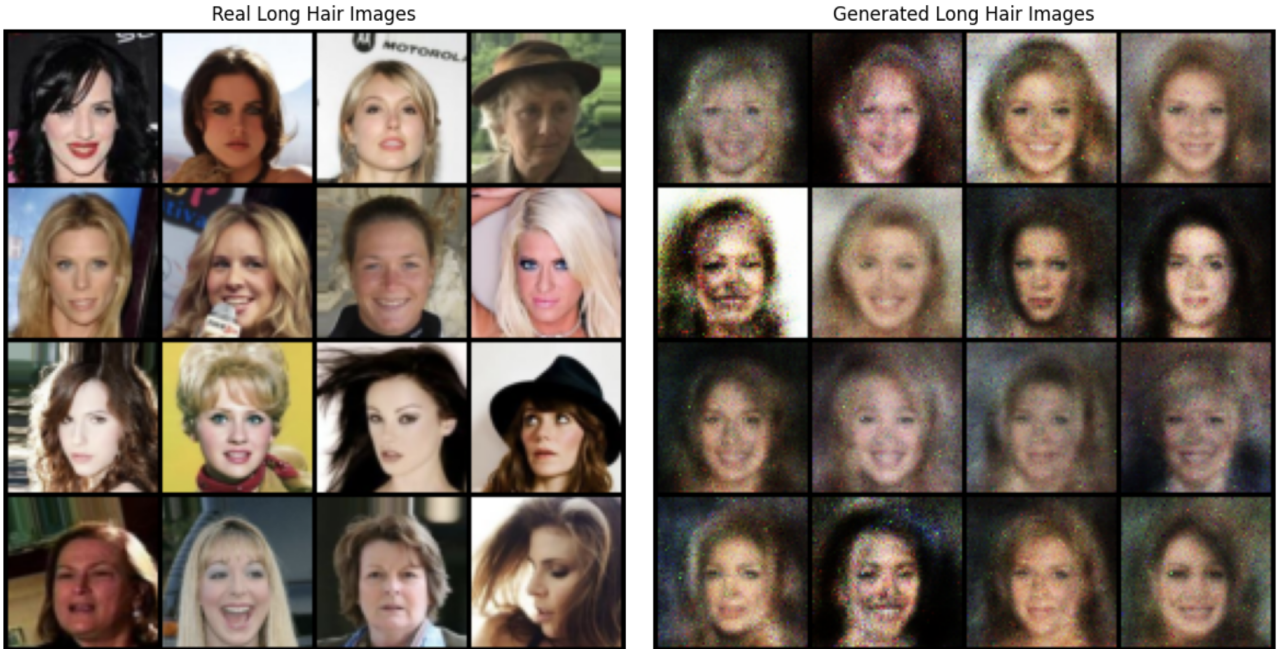


Figure 3: Real vs. generated face images with long hair.

The training loss curves of the generator and discriminator over epochs are shown in Figure 4, indicating the adversarial learning progression and relative convergence.

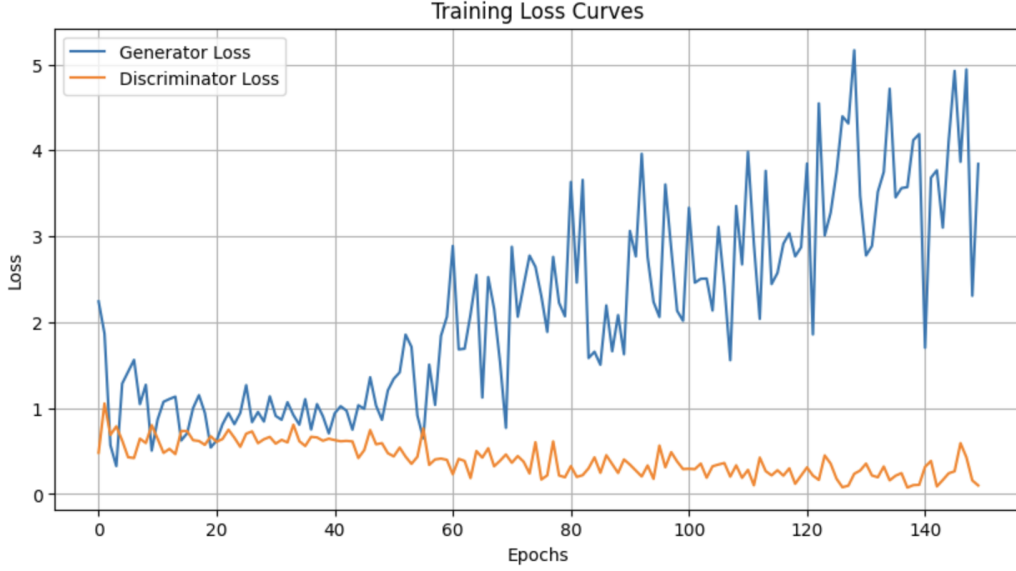


Figure 4: Generator and discriminator loss during CGAN training.

2.4 Conclusion

The Conditional GAN effectively generated realistic synthetic faces conditioned on long hair presence. While visual quality was promising, some artifacts remained, motivating further improvements such as architecture tuning or additional training.

3 Reason for Using BCE Loss Instead of Minimax Loss in TensorFlow Implementation

3.1 Original Minimax Loss Formulation

The original GAN framework, introduced by Goodfellow et al. [1], is defined as a two-player minimax game between the generator G and the discriminator D :

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \quad (1)$$

Here, $D(x)$ represents the discriminator's probability that x is a real image, while $G(z)$ maps a random noise vector z to a synthetic image.

3.2 The Problem with Minimax Loss for the Generator

In the minimax setting, the generator minimizes:

$$\mathcal{L}_G^{\text{minimax}} = \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \quad (2)$$

When the discriminator becomes strong early in training, $D(G(z)) \approx 0$. In such cases:

$$\log(1 - D(G(z))) \approx \log(1) = 0$$

and the gradient magnitude becomes very small:

$$\frac{\partial}{\partial \theta_G} \log(1 - D(G(z))) = -\frac{1}{1 - D(G(z))} \cdot \frac{\partial D(G(z))}{\partial \theta_G} \quad (3)$$

This leads to the *vanishing gradient* problem, where the generator receives almost no learning signal during the early stages of training.

3.3 Non-Saturating Loss (BCE) as a Solution

To address this, Goodfellow et al. [1] proposed an alternative objective, known as the *non-saturating* loss, where the generator maximizes $\log D(G(z))$ instead of minimizing $\log(1 - D(G(z)))$:

$$\mathcal{L}_G^{\text{non-sat}} = -\mathbb{E}_{z \sim p_z} [\log D(G(z))] \quad (4)$$

This is equivalent to using a binary cross-entropy (BCE) loss with target label 1 for generated samples:

$$\text{BCE}(D(G(z)), 1) = -\log D(G(z))$$

The gradient of this loss is:

$$\frac{\partial}{\partial \theta_G} [-\log D(G(z))] = -\frac{1}{D(G(z))} \cdot \frac{\partial D(G(z))}{\partial \theta_G} \quad (5)$$

When $D(G(z)) \approx 0$, the term $\frac{1}{D(G(z))}$ becomes large, resulting in strong gradient signals and allowing the generator to learn effectively from the start.

3.4 TensorFlow Implementation Choice

TensorFlow’s official DCGAN implementation [2] uses `tf.keras.losses.BinaryCrossentropy(from_logits=True)` for both generator and discriminator losses. Specifically:

- For the discriminator: real images $\rightarrow$ label 1, fake images $\rightarrow$ label 0.
- For the generator: fake images $\rightarrow$ label 1 (non-saturating BCE loss).

This choice follows the recommendation in the original GAN paper’s Section 3, avoiding the vanishing gradient issue inherent in the minimax formulation.

3.5 References

References

- [1] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. *Generative Adversarial Nets*. Advances in Neural Information Processing Systems, 2014.
- [2] TensorFlow Developers. *Deep Convolutional Generative Adversarial Network*. TensorFlow Tutorials. Available at: <https://www.tensorflow.org/tutorials/generative/dcgan>
- [3] Alec Radford, Luke Metz, and Soumith Chintala. *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*. arXiv preprint arXiv:1511.06434, 2015.